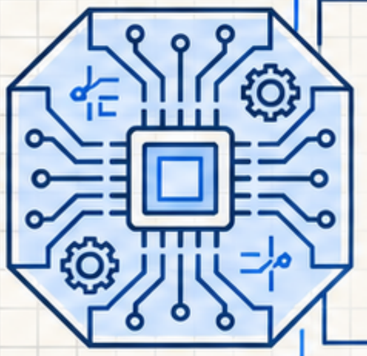
Deep Agents 实战

v0.7 内容校准版

第 2 章：快速上手

构建你的第一个 Deep Agent



```
from deepagents import create_deep_agent
create_deep_agent(...)
agent.invoke(...)
```

安装配置 → 自定义工具 → 研究助手 → 运行追踪

安装与配置

一条一致的硅基流动接入路径



快速接入　模型可选　统一接口

Python 3.11+

```
pip install deepagents langchain-openai
```

设置环境变量（示例）

```
export SILICONFLOW_API_KEY="your-key"
export MODEL_NAME="Qwen/Qwen2.5-7B-Instruct"
```

示例代码：创建模型实例

```
import os
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model=os.environ["MODEL_NAME"],
    api_key=os.environ["SILICONFLOW_API_KEY"],
    base_url="https://api.siliconflow.cn/v1",
)
```

后续示例复用此 model；密钥从环境变量读取，不写入源码。

课程 Notebook 使用单独的 Python 3.12 锁定环境。

模型接入：三种方式

一张表看懂所有接入方法

方式	代码	适用场景
推荐 ChatOpenAI + base_url（推荐）	`ChatOpenAI(model=..., base_url="https://api.siliconflow.cn/v1")`	第三方平台（硅基流动等）
ChatAnthropic + base_url	`ChatAnthropic(model=..., base_url="https://api.siliconflow.cn")`	Anthropic 兼容平台
字符串格式	`create_deep_agent(model="openai:gpt-4.1")`	官方 API 直连

行业两大标准：OpenAI 兼容（/chat/completions）和 Anthropic 兼容（/messages），国内平台几乎都支持 OpenAI 兼容

模型选择：先跑通，再升级

模型须支持工具调用；用 MODEL_NAME 管理切换

入门与简单工具

`Qwen/Qwen2.5-7B-Instruct`

- 章节中的轻量示例模型
- 先验证单次工具调用与输入输出

复杂研究与多步编排

`zai-org/GLM-5.2`

- 章节建议使用更强的工具调用模型
- 规划、总结、委派需要更稳定的能力

⚠ 模型名称、价格、免费额度和限流会变化。运行前查看提供商模型页，不把模型规格写成永久保证。

ⓘ 这是章节选型示例，不是本次性能测试结果。

Hello World + 核心参数

复用安装配置中的 model，创建一个工具调用示例

```python
from deepagents import create_deep_agent

def get_weather(city: str) -> str:
    "Demo weather."
    return f"It's always sunny in {city}!"

agent = create_deep_agent(
    model=model,
    tools=[get_weather],
    system_prompt="You are a helpful assistant.",
)
result = agent.invoke({"messages": [
    {"role": "user", "content": "北京天气怎么样？"}
]})
print(result["messages"][-1].content)
```

model
模型实例

tools
自定义工具

system_prompt
角色与行为

注意： get_weather 返回模拟天气，不是实时天气查询。

v0.7： Todo 规划不是默认能力，需显式启用。

工具定义三要素

一个 Python 函数就是一个工具

- 参数类型标注 — 告诉 Agent 每个参数该传什么类型
- Docstring — 告诉 Agent 这个工具的用途（'使用说明书'）
- 默认值 — 标记可选参数，减少 Agent 出错概率

```python
def internet_search(
    query: str,                 # 类型标注
    max_results: int = 5,       # 默认值
) -> dict:
    """Run a web search."""     # Docstring
```

① 参数类型标注与默认值

② 默认值

③ Docstring

要素	对 Agent 的作用
① 参数类型标注	明确输入数据格式，提高调用准确性
② 默认值	简化调用，处理非必要参数，降低错误率
③ Docstring	提供工具用途和使用说明，指导 Agent 正确决策

实战：构建研究助手

目标、工具与规划能力分别配置

1

搜索工具

Tavily + internet_search

2

研究目标

问题：什么是 LangGraph？

3

创建 Agent

复用已配置的 model

4

执行与检查

查看报告与引用来源

```
from langchain.agents.middleware import TodoListMiddleware

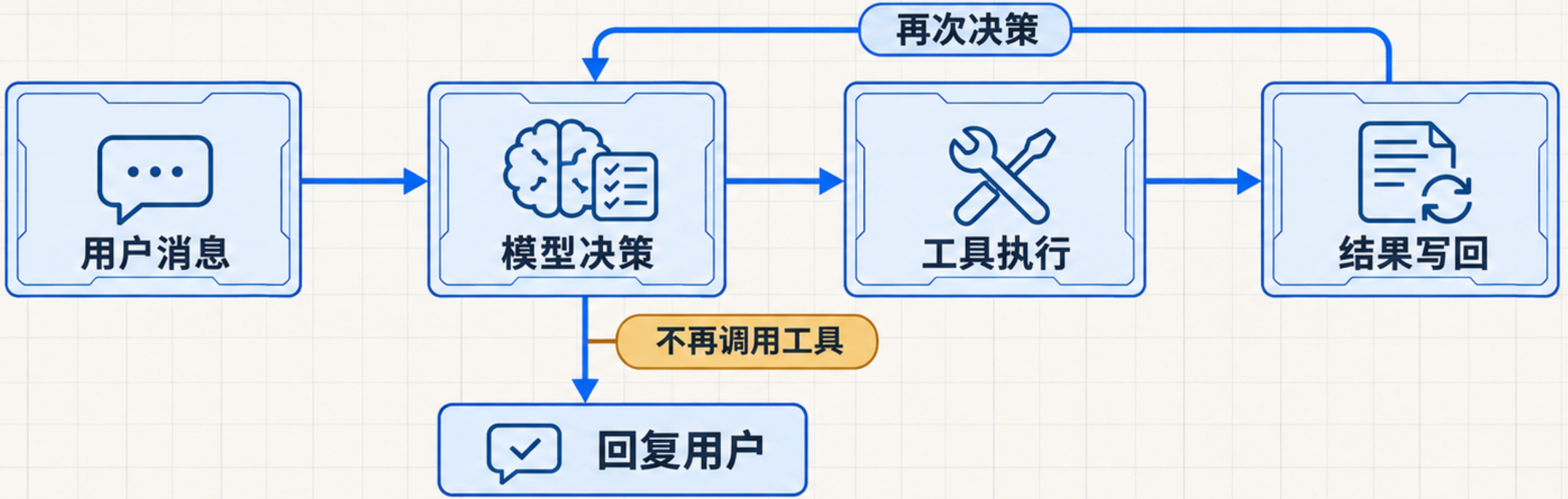
agent = create_deep_agent(
    model=model,
    tools=[internet_search],
    system_prompt=research_instructions,
    middleware=[TodoListMiddleware()],
)
```

- 额外安装 tavily-python
- 配置 TAVILY_API_KEY
- 工具和提示词定义见章节完整代码

此页展示装配方式，不把示意文字冒充真实运行结果。

agent.invoke() 背后发生了什么？

模型—工具循环，不是固定的五步流水线



可选规划

已启用 Todo 后，模型可选择 `write_todos`

可选委派

需要时通过 `task` 调用子 Agent

按配置暂停

人工审批中断后，等待恢复

调用次数、顺序和是否委派由任务与模型决定；不保证“10+ 次”。

LangSmith 调试与追踪

记录模型和工具调用，不等于读取模型的隐藏思考

US 区域示例（.env）

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=your-key
LANGSMITH_PROJECT=deepagents-course
LANGSMITH_ENDPOINT=https://api.smith.langchain.com
```

加载环境变量

.env 不会被普通 Python 自动读取

区域保持一致

Key、API Endpoint、查看控制台对应同一区域

查看运行轨迹

模型输入输出、工具参数结果、耗时与 Token

EU / APAC 请使用章节中的对应 Endpoint；更新配置后重启运行进程。

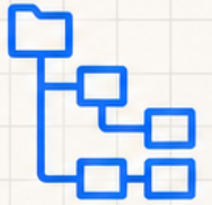
模型 base_url 与追踪 API Endpoint 是不同配置。

本章回顾

- 环境搭建 / 安装依赖，配置 `SILICONFLOW_API_KEY`
- 模型接入 / `ChatOpenAI` + 明确的 `api_key` 与 `base_url`
- 核心 API / `create_deep_agent()` 创建，`invoke()` 运行
- 能力选择 / 文件与子 Agent 能力；`Todo` 需显式启用
- 调试验证 / 检查工具结果与追踪配置，而非只看最终回答

→ 下一章：虚拟文件系统与上下文工程

</> 配套代码与细节：第 2 章文章 + 两个 Notebook



ⓘ 本版对齐课程 v0.7 说明；未执行外部模型 API 实测。